

Morphic Studio Product Architecture Specification

Document 51 — Product Module Architecture & Capability Map

Version: 1.0

Status: Approved

1. Purpose

This document defines the complete product module architecture for Morphic Studio.

Its purpose is to identify every major user-facing capability, establish module boundaries, define ownership and responsibilities, and describe how modules collaborate to provide a unified creative experience.

This document serves as the master capability map for the entire platform.

2. Product Philosophy

Every module should satisfy the following principles:

- Solve a complete user problem rather than exposing isolated features.
 - Integrate naturally with every other module.
 - Remain independently maintainable.
 - Share common platform services.
 - Support AI assistance where appropriate.
 - Respect permissions, security, and project context.
 - Be extensible through the Plugin Framework.
-

3. Platform Capability Domains

The platform is organized into the following capability domains:

Creative Writing

Responsible for narrative creation and editing.

Includes:

- Story Workspace
 - Chapter Editor
 - Scene Editor
 - Dialogue Editor
 - Outline Builder
-

Character Creation

Responsible for character design and management.

Includes:

- Character Studio
 - Relationships
 - Character Memory
 - Character Evolution
 - Character Gallery
-

World Building

Responsible for fictional universe management.

Includes:

- Locations
 - Organizations
 - Cultures
 - Technology
 - Magic Systems
 - Geography
 - History
-

Story Bible

Maintains canonical project knowledge.

Includes:

- Canon Rules
- Timeline
- Lore

- Character Registry
 - World Registry
 - Glossary
 - Version History
-

AI Creative Assistant

Provides intelligent assistance.

Includes:

- AI Chat
 - Prompt Workspace
 - Context Explorer
 - AI Recommendations
 - Multi-Agent Collaboration
 - Prompt History
-

Visual Creation

Responsible for image-based creative work.

Includes:

- Concept Art
 - Character Art
 - Storyboards
 - Comic Panels
 - Scene Composition
 - Visual References
-

Animation Studio

Supports motion production.

Includes:

- Animation Planning
 - Scene Sequencing
 - Motion Design
 - Shot Planning
 - Timing
-

Audio Studio

Responsible for sound production.

Includes:

- Voice Generation
 - Narration
 - Sound Effects
 - Music Planning
 - Audio Assets
-

Video Studio

Responsible for video production.

Includes:

- Timeline Editing
 - Scene Assembly
 - Rendering
 - Subtitle Management
 - Video Export
-

Asset Management

Maintains all creative assets.

Supports:

- Images
 - Audio
 - Video
 - Documents
 - Models
 - References
 - Generated Content
-

Production Management

Coordinates project execution.

Includes:

- Production Board
 - Tasks
 - Milestones
 - Schedules
 - Approvals
 - Workflow Tracking
-

Publishing Center

Responsible for distribution.

Supports:

- Export
 - Publishing
 - Versioning
 - Distribution Targets
 - Release Management
-

Collaboration

Enables teamwork.

Supports:

- Comments
 - Reviews
 - Mentions
 - Sharing
 - Permissions
 - Team Activity
-

Analytics

Provides insights.

Includes:

- Productivity
- AI Usage
- Workflow Analytics

- Project Health
 - Cost Analysis
-

Automation

Coordinates workflow execution.

Includes:

- Workflow Builder
 - Scheduled Jobs
 - Templates
 - AI Automation
 - Event Triggers
-

Marketplace

Supports ecosystem expansion.

Includes:

- Plugin Discovery
 - Installations
 - Updates
 - Ratings
 - Publisher Profiles
-

Administration

Provides platform administration.

Supports:

- User Management
 - Organizations
 - Billing
 - Security
 - Audit Logs
 - Compliance
-

Developer Tools

Supports advanced users and integrators.

Includes:

- API Explorer
 - SDK Tools
 - Plugin Development
 - Debug Utilities
 - Diagnostics
-

4. Shared Platform Services

All modules consume shared platform services including:

- Authentication
- Authorization
- AI Gateway
- Knowledge Graph
- Creative Memory
- Asset Storage
- Search
- Notifications
- Workflow Engine
- Telemetry
- Plugin Framework

Shared services eliminate duplication and provide consistent behavior.

5. Cross-Module Communication

Modules communicate through documented APIs and platform events.

Direct internal dependencies should be minimized.

All inter-module communication should be observable and versioned.

6. Module Lifecycle

Each module progresses through the following lifecycle:

Concept



Architecture



Implementation



Testing



Release



Monitoring



Iteration



Maintenance



Retirement (if necessary)

Every stage should be documented and traceable.

7. Capability Ownership

Every capability must define:

- Functional owner
- Technical owner
- Public API

- Dependencies
- Permissions
- Data ownership
- AI integration points
- Extension points
- Operational metrics

Clear ownership reduces ambiguity and simplifies maintenance.

8. AI Integration Across Modules

Every major module may expose AI-assisted capabilities.

Examples include:

- Story suggestions
- Character consistency checks
- World-building recommendations
- Visual generation
- Audio generation
- Workflow optimization
- Publishing assistance

AI interactions should always pass through the Unified AI Gateway.

9. Extensibility

Each module should define supported extension points for:

- Plugins
- Marketplace integrations
- Enterprise customizations
- Workflow actions
- Automation triggers
- External APIs

Modules should remain open for extension but closed for modification.

10. Acceptance Criteria

The product architecture is successful if:

- Every user-facing capability belongs to a clearly defined module.
 - Module responsibilities do not overlap unnecessarily.
 - Shared services provide common functionality.
 - Cross-module communication is standardized.
 - AI integration is consistent across the platform.
 - Extension points are documented.
 - Future capabilities can be added without restructuring the platform.
-

Conclusion

The Morphic Studio Product Module Architecture establishes a comprehensive capability map that organizes the platform into cohesive, interoperable modules supported by shared services and consistent architectural principles. This structure enables scalable product evolution, simplifies development, and provides users with an integrated creative environment rather than a collection of disconnected tools.